Патерни, тренди та кластеризація: Neo4j і Gephi на даних Bitcoin

Заняття №13

AI для OSINT і розвідки

r_d

ПЛАН ЗАНЯТТЯ



- Від воркшопу до масштабу: три питання
- Дані: BitcoinTemporalGraph та Elliptic++
- Neo4j: імпорт, Cypher-патерни, GDS
- Кластеризація: евристики та спільноти
- Тренди: час, дрейф, закриття ринку
- Gephi: від підграфа до висновку
- Обмеження, етика та практика

ВІД ВОРКШОПУ ДО МАСШТАБУ

Минулого заняття — воркшоп

- Ручне трасування в оглядачах і графах аналітичних платформ
- Peeling-ланцюги, пулінг, адреси-«сховища» міксерів
- Кейс A7A5: Garantex → A7A5 → Grinex
- Neo4j згадали як інструмент — сьогодні працюємо в ньому

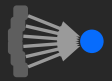
Сьогодні

- 822 942 гаманці замість десятка адрес
- Патерн = запит, який можна повторити
- Кластер = алгоритм із параметрами, а не «на око»
- Тренд = метрика в часі + перевірка дрейфу
- Візуалізація в Gephi як аргумент у звіті

Мета заняття: перетворити інтуїцію з воркшопу на відтворювані запити, метрики та візуалізації.

ТРИ ПИТАННЯ АНАЛІТИКА

ПАТЕРН



Як рухаються кошти?

- Fan-in, fan-out, peel chain, цикли, CoinJoin
- Структура, а не окрема транзакція

Інструмент

Cypher-шаблони шляхів

ТРЕНД



Що змінюється в часі?

- Обсяги illicit, нові вузли, зникнення сервісів
- Дрейф після подій

Інструмент

Агрегації за кроком часу, Timeline у Gephi

КЛАСТЕР



Хто діє разом?

- Евристики адрес, спільноти, схожа поведінка
- Групи як одиниця аналізу

Інструмент

GDS: WCC, Louvain, Leiden, k-means

Кожна відповідь — гіпотеза для офчейн-перевірки, а не вирок.

ЧАСТИНА 1

Дані: два датасети



- BitcoinTemporalGraph: як будують граф
- Формула ребра на прикладі
- OSINT-розмітка: BitcoinTalk + LLM
- Мітки: джерела та розподіл
- Критичне читання статті
- Elliptic++: файли, мітки, час

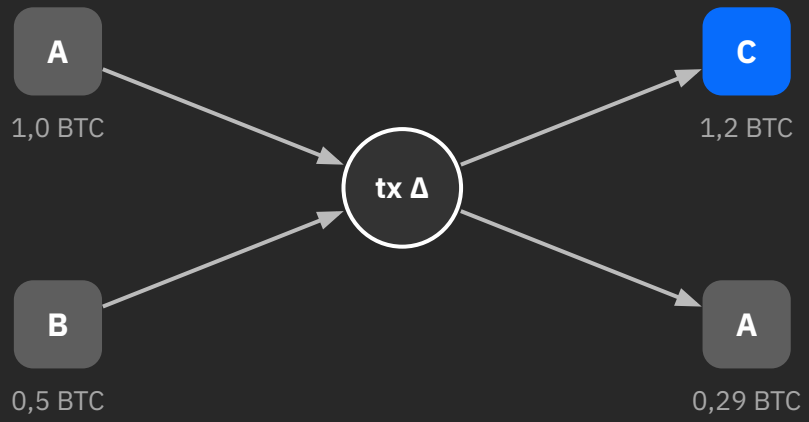
ДВА ДАТАСЕТИ

	BitcoinTemporalGraph	Elliptic++
Джерело	Schnoering & Vazirgiannis, arXiv 2411.10325 (2024)	Elmougy & Liu, KDD ’23 (Georgia Tech)
Вузол	Сутність = кластер скриптів (адрес)	Транзакція; гаманець (адреса)
Ребро	Переказ вартості між сутностями	tx → tx; адреса → адреса; адреса → tx → адреса
Масштаб	252,2 млн вузлів · 786 млн ребер · 670 млн tx	203 769 tx · 822 942 гаманці · 2,87 млн ребер addr→addr
Час	Індекс блоку, перші 700 000 блоків (~13 років)	49 кроків, ~2 тижні між ними
Мітки	34 098 вузлів, 11 типів сутностей	illicit / licit / unknown
Формат	Дамп PostgreSQL: ~120 ГБ після відновлення (CC BY 4.0)	CSV, близько 1,6 ГБ
На занятті	Методологія, OSINT-розмітка, бенчмарк	Практика: Neo4j + Gephi

ЯК ПОБУДОВАНО BITCOINTEMPORALGRAPH

1. Сирі дані	Вузол Bitcoin Core → файли blk*.dat; перші 700 000 блоків, ~13 років історії
2. Скрипти	Кожен вихід (TXO) захищений locking script; нульові TXO відкинуто → понад 874 млн скриптів
3. Кластери	Евристики об’єднують скрипти одного власника → ~252 млн сутностей, це і є вузли
4. Ребра	Чиста сума v(a) = отримано − витрачено: v > 0 — отримувач, v < 0 — відправник
5. Фільтри	CoinJoin (JoinMarket, Wasabi, Whirlpool) і colored coins (Omni, Open Assets, EPOBC) виключено
6. Атрибути	Ребро: reveal, last_seen, total, min/max/total_sent. Вузол: ступені, суми, часові межі, розмір кластера

ФОРМУЛА РЕБРА НА ПРИКЛАДІ



Комісія майнеру: 0,01 BTC. A отримує 0,29 BTC решти

v(a) = Σ виходів a − Σ входів a

- v(A) = 0,29 − 1,0 = −0,71 → відправник
- v(B) = 0 − 0,5 = −0,5 → відправник
- v(C) = 1,2 − 0 = +1,2 → отримувач
- Частки входів: A = 1,0 / 1,5 = ⅔, B = ⅓
- Ребра: A → C = ⅔ × 1,2 = **0,8 BTC**; B → C = ⅓ × 1,2 = **0,4 BTC**
- Решта A → A не створює ребра

Модельне рішення: розподіл за часткою входів. Інша модель дала б інші ваги — фіксуйте це в методології.

OSINT-РОЗМІТКА: BITCOINTALK + LLM

BitcoinTalk
14 067 713 постів
546 440 тем (англ.)

→

Контекст
Тема, пост, деталі tx
+ курс USD на дату

→

gpt-4o-mini
4 типи промптів:
депозит, виведення, hot/cold

→

Відповідь
Міркування, назва сутності, тип, адреси

→

Мітка вузла
Назва → категорія; конфлікт → без мітки

Сценарії промптів

- Депозит: користувач скаржиться, що кошти не зараховано → адреса депозиту належить сервісу
- Виведення: сервіс повідомив txid → входи tx = гарячий гаманець сервісу
- Виведення без txid: пошук tx ±3 дні на адресу користувача з указаною сумою; якщо одна — атрибуція

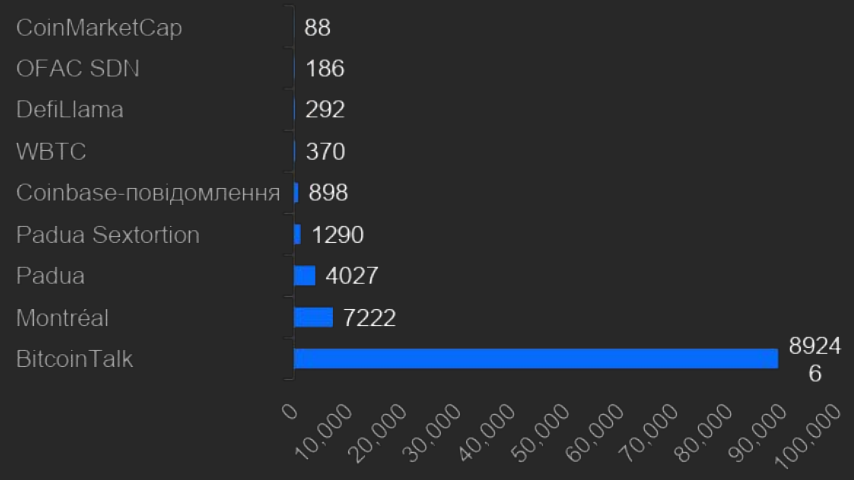
Як мітка доходить до вузла

- Назву сутності мапують на 11 категорій (форум, інтернет, Wayback Machine)
- Адреса → усі скрипти, які вона відмикає → кластер (вузол)
- Кілька різних міток в одному кластері → мітку не ставлять

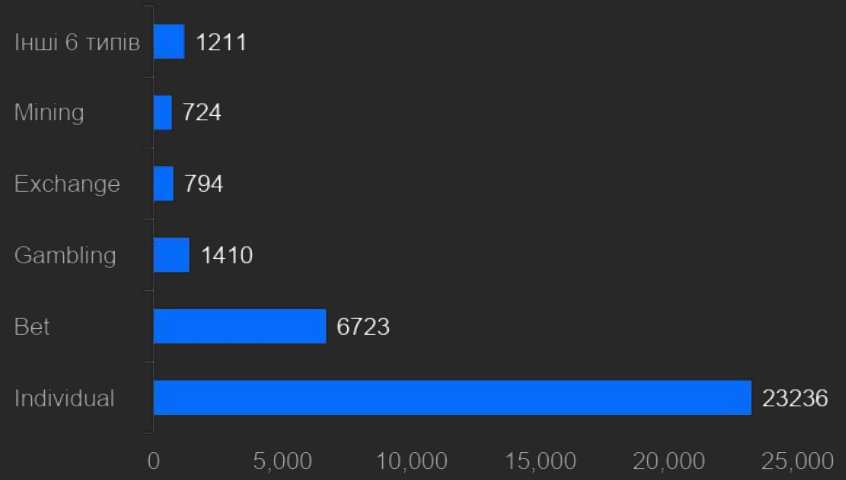
Це класичний OSINT: офчейн-контекст (скарги на форумі, повідомлення сервісів) дає атрибуцію ончейн-адрес.

МІТКИ: ДЖЕРЕЛА ТА РОЗПОДІЛ

Джерела міток адрес (рядки таблиці 4)



Типи сутностей серед вузлів з мітками



34 098 вузлів з мітками = 0,014% графа (≈ 1 вузол із 7 400). Мітки рідкісні — сигнал треба брати зі структури.

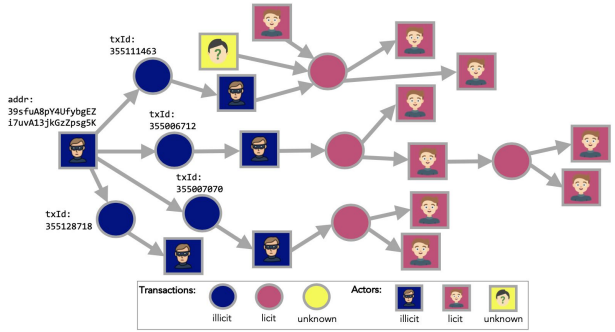
ЧИТАЄМО СТАТТЮ ЯК OSINT-ДЖЕРЕЛО

Що в статті	Перевірка	Висновок
Щільність графа «≈ 1%»	785 954 737 / (252 219 007 × 252 219 006) ≈ $1{,}2 \times 10^{-8}$	Помилка на ~6 порядків; середній ступінь ≈ 6,2
«101 186 адрес з мітками»	Сума рядків табл. 3 і табл. 4 = 103 619; відсотки рахують саме від неї	Розбіжність у 2 433 адреси не пояснено
«SDN List веде Мін’юст США»	Список SDN веде OFAC (Мінфін США); посилання статті — на sanctionslist.ofac.treas.gov	Неточна атрибуція джерела
Табл. 1: b — «alias of the sender»	Ребро a → b: b має бути отримувачем	Опечатка в схемі даних

Препринт — це джерело, а не істина: перераховуйте ключові цифри, перш ніж цитувати їх у звіті.

ELLIPTIC++: ЩО ВСЕРЕДИНІ

Файл	Зміст	Рядків
txs_features.csv	183 ознаки: крок часу + 93 local + 72 aggregated + 17 augmented	203 769
txs_classes.csv	Клас: 1 illicit · 2 licit · 3 unknown	203 769
txs_edgelist.csv	tx → tx (потік коштів)	234 355
wallets_features.csv	56 ознак гаманця, рядок = гаманець × крок	1 268 260
wallets_classes.csv	Клас гаманця	822 942
AddrAddr_edgelist.csv	Адреса → адреса	2 868 964
AddrTx_edgelist.csv	Адреса → tx (вхід)	477 117
TxAddr_edgelist.csv	tx → адреса (вихід)	837 124



Граф адреса → tx → адреса для illicit-гаманця 39sfuA...5K.
Рисунок: Elmougy & Liu, Elliptic++

ELLIPTIC++: МІТКИ ТА ЧАС

Транзакції (203 769)

4 545	42 019	157 205
illicit · 2,2%	licit · 20,6%	unknown · 77,1%

Гаманці (822 942)

14 266	251 088	557 588
illicit · 1,7%	licit · 30,5%	unknown · 67,8%

Час

- 49 кроків, ~2 тижні між ними
- Крок = транзакції, підтверджені в межах < 3 годин
- Перевірено: 49 компонент = 49 кроків, ребер між кроками — 0
- Гаманець: перший і останній крок появи

unknown ≠ licit: три з чотирьох транзакцій не розмічені — це простір для гіпотез, а не «чисті» дані.

ЧАСТИНА 2

Neo4j: від CSV до патернів



- Чому графова база даних
- Встановлення: Desktop або Docker
- Модель даних і підготовка CSV
- Імпорт Elliptic++
- Галерея патернів і сигнатури
- Cypher-запити та Graph Data Science

ЧОМУ ГРАФОВА БАЗА ДАНИХ

SQL (PostgreSQL): рекурсивний CTE

```
WITH RECURSIVE hop(addr, d) AS (
  SELECT output_address, 1 FROM addr_addr
  WHERE input_address = '1Abc...'
  UNION
  SELECT e.output_address, h.d + 1
  FROM addr_addr e
  JOIN hop h ON e.input_address = h.addr
  WHERE h.d < 3
)
SELECT DISTINCT addr FROM hop;
```

Cypher (Neo4j): той самий запит

```
MATCH (:Wallet {address: '1Abc...'})
      -[:SENT_TO*1..3]->(w:Wallet)
RETURN DISTINCT w.address;

// Гаманці в межах 3 кроків
// від заданої адреси
```

PostgreSQL (як у статті): масове сховище на 120 ГБ, вибірки, агрегати

Neo4j: багатокрокові шляхи, патерни, алгоритми GDS, швидка візуальна перевірка

Практичне правило: велике сховище — у SQL, підграф розслідування — у Neo4j.

NEO4J: ВСТАНОВЛЕННЯ

Версії (листопад 2026)

- Neo4j 2026.10.1 — Cypher 25 за замовчуванням
- GDS і APOC 2026.10.1 вже в дистрибутиві: products/ і labs/

Neo4j Desktop

- База → Plugins → APOC, Graph Data Science → Install

Tarball або Docker

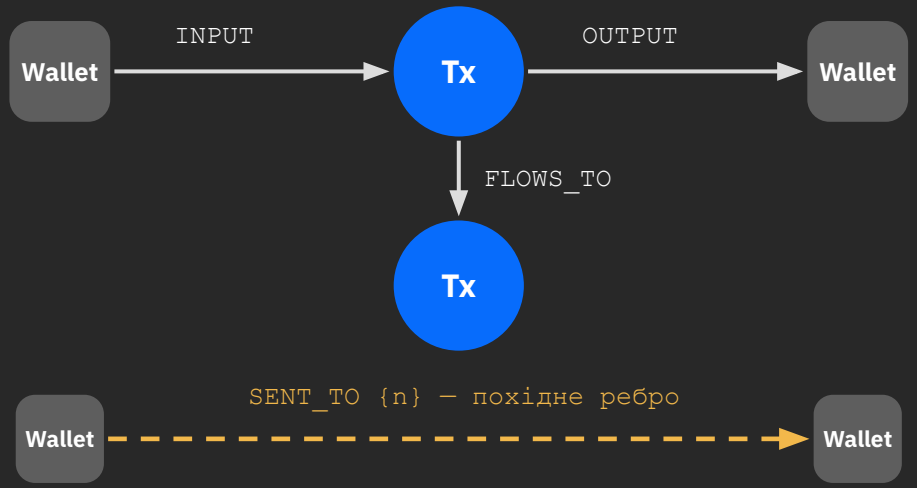
- Скопіювати jar-файли в plugins/ або команда праворуч

```
docker run -d --name elliptic \
  -p 7474:7474 -p 7687:7687 \
  -v $PWD/import:/import \
  -v $PWD/data:/data \
  -e NEO4J_AUTH=neo4j/osint2026pass \
  -e NEO4J_PLUGINS='["apoc","graph-data-science"]' \
  -e NEO4J_apoc_export_file_enabled=true \
  -e NEO4J_server_memory_heap_max__size=4G \
  -e NEO4J_server_memory_pagecache_size=2G \
  neo4j:2026.10.1

# Браузер: http://localhost:7474
```

Перевірка: RETURN gds.version(), apoc.version(); · У Neo4j Browser запити з CALL … IN TRANSACTIONS запускайте з префіксом :auto

МОДЕЛЬ ДАНИХ У NEO4J




```
(:Transaction)
  txId, timeStep, class
  nIn, nOut, totalBTC
  fees, outMin, outMax

(:Wallet)
  address, class
  firstStep, lastStep

[:SENT_TO]   n — скільки разів
             пара зустрілась
```

Один датасет — три графи: потік коштів (tx → tx), взаємодія гаманців (addr → addr), двочастковий (addr → tx → addr).

ПІДГОТОВКА CSV У PANDAS

```
import pandas as pd
cols = ['txId', 'Time step', 'num_input_addresses',
        'num_output_addresses', 'total_BTC', 'fees',
        'out_BTC_min', 'out_BTC_max', ...]
tx = pd.read_csv(SRC + 'txs_features.csv', usecols=cols)
tx = tx.merge(pd.read_csv(SRC + 'txs_classes.csv'), on='txId')
tx.rename(columns={'Time step': 'timeStep'}) \
  .to_csv(OUT + 'tx_nodes.csv', index=False)

w = pd.read_csv(SRC + 'wallets_features.csv',
                usecols=['address', 'Time step'])
w = w.groupby('address')['Time step'] \
     .agg(firstStep='min', lastStep='max').reset_index()

aa = pd.read_csv(SRC + 'AddrAddr_edgelist.csv')
aa = aa[aa.input_address != aa.output_address]
aa = aa.groupby(['input_address', 'output_address']) \
       .size().reset_index(name='n')
```

Що робимо і навіщо

- 10 зі 184 колонок: 695 МБ → 14 МБ
- Гаманець: один рядок + перший і останній крок
- Self-loops (решта на ту саму адресу): прибрано 45 981
- Дублікати пар → вага n: 2 822 983 → 2 775 637 пар, n до 87

Гігієна даних: кожне перетворення — у скрипті поруч з даними, а не вручну в таблиці.

ІМПОРТ: LOAD CSV

```
CREATE CONSTRAINT tx_id IF NOT EXISTS
  FOR (t:Transaction) REQUIRE t.txId IS UNIQUE;
CREATE CONSTRAINT wallet_addr IF NOT EXISTS
  FOR (w:Wallet) REQUIRE w.address IS UNIQUE;

LOAD CSV WITH HEADERS FROM 'file:///tx_nodes.csv' AS row
CALL (row) {
  CREATE (:Transaction {txId: toInteger(row.txId),
    timeStep: toInteger(row.timeStep),
    class: toInteger(row.class),
    nOut: toInteger(row.num_output_addresses), ...})
} IN TRANSACTIONS OF 10000 ROWS;

LOAD CSV WITH HEADERS FROM 'file:///sent_to.csv' AS row
CALL (row) {
  MATCH (a:Wallet {address: row.input_address})
  MATCH (b:Wallet {address: row.output_address})
  CREATE (a)-[:SENT_TO {n: toInteger(row.n)}]->(b)
} IN TRANSACTIONS OF 20000 ROWS;
```

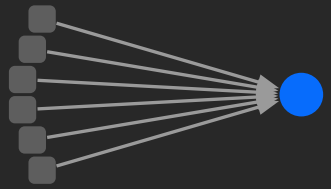
Результат перевірки

Об’єкт	Кількість
Transaction	203 769
Wallet	822 942
FLOWS_TO	234 355
INPUT	477 117
OUTPUT	837 124
SENT_TO	2 775 637

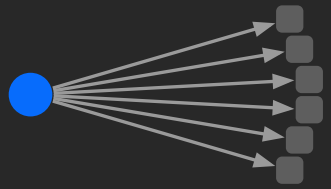
Імпорт: 4 хв 17 с на 2 CPU / 7 ГБ RAM; найдовше — SENT_TO (147 с)

Обмеження унікальності — до імпорту: інакше кожен MATCH сканує всі вузли. Повний скрипт — 01_import.cypher.

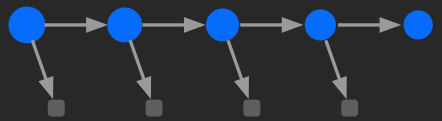
ГАЛЕРЕЯ ПАТЕРНІВ



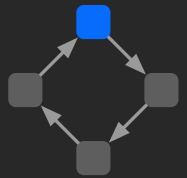
Fan-in · консолідація
Багато відправників → 1 отримувач



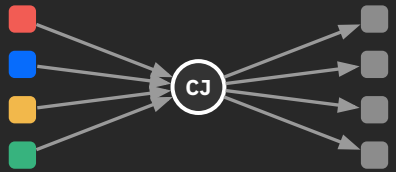
Fan-out · розпилення
1 відправник → багато отримувачів



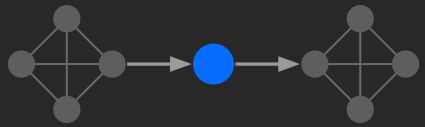
Peel chain
Ланцюг tx: малий «відщеп» + решта далі



Цикл · round-trip
Кошти повертаються до старту



CoinJoin
Входи різних власників → рівні виходи



Міст · хаб
Вузол між двома щільними групами

ПАТЕРН → СИГНАТУРА → ЗАПИТ

Патерн	Сигнатура в даних	Інструмент	Легітимне пояснення
Fan-in	Вхідний ступінь ≫ медіани	count() / gds.degree REVERSE	Біржа, пул майнерів, донати
Fan-out	Вихідний ступінь ≫ медіани, nOut велике	count() / nOut	Пакетні виплати бірж, зарплати
Peel chain	≥ 5 tx поспіль: 2 виходи, малий < 10%	Шлях *5..8 + all()	Звичайний гаманець з рештою
Цикл	Шлях повертається до старту за 2–4 кроки	(w)-[*2..4]->(w)	Перекази між власними гаманцями
CoinJoin	Багато входів, рівні виходи	Фільтр nIn, nOut, outMin ≈ outMax	Приватність, не злочин
Міст	Висока betweenness між спільнотами	gds.betweenness + Louvain	Біржа, платіжний шлюз

Патерн — гіпотеза: у кожного є легітимне пояснення, яке треба виключити до висновку.

CYPHER: КОНСОЛІДАЦІЯ ТА ЕКСПОЗИЦІЯ

Fan-in: хто збирає кошти

```
MATCH (s:Wallet)-[:SENT_TO]->(w:Wallet)
WITH w, count(s) AS senders
WHERE senders >= 100
RETURN w.address, w.class, senders
ORDER BY senders DESC LIMIT 10;
```

Експозиція: куди доходять кошти illicit

```
MATCH (i:Wallet {class: 1})
      -[:SENT_TO*1..2]->(u:Wallet {class: 3})
WITH u, count(DISTINCT i) AS exposure
WHERE exposure >= 5
RETURN u.address, exposure
ORDER BY exposure DESC LIMIT 10;
```

Результат на Elliptic++ (4,3 с): ≥ 100 відправників мають 2 680 гаманців — 2 078 licit, 578 unknown, лише 24 illicit

Топ-1: 11 213 відправників, licit. Fan-in — передусім сервіси, а не злочин

Результат (0,5 с): два unknown-гаманці отримують кошти від 1 061 illicit-гаманця кожен, далі — 319 і 190

Однакові значення — підказка, що це адреси одного сервісу: перевірте fan-in і офчейн

CYPHER: PEEL CHAIN, ЦИКЛИ, ШЛЯХИ

```
// Peel chain: >= 5 tx поспіль з 2 виходами, «відщеп» < 10% від більшого виходу
MATCH p = (s:Transaction {class: 1})-[:FLOWS_TO*5..8]->(:Transaction)
WHERE all(t IN nodes(p) WHERE t.nOut = 2 AND t.outMin < 0.1 * t.outMax)
RETURN length(p) AS hops, [t IN nodes(p) | t.txId] AS chain LIMIT 5;
```

```
// Цикли: повернення за 2–4 кроки
MATCH p = (w:Wallet {class: 1})
          -[:SENT_TO*2..4]->(w)
RETURN [x IN nodes(p) | x.address] LIMIT 5;
```

```
// Найкоротший шлях між двома гаманцями
MATCH (a:Wallet {address: $from})
MATCH (b:Wallet {address: $to})
MATCH p = shortestPath((a)-[:SENT_TO*..8]->(b))
RETURN [x IN nodes(p) | x.address], length(p);
```

Peel chain (0,14 с): умову «2 виходи, малий < 10%» задовольняють 102 777 tx — половина датасету; ланцюг ≥ 5 стартує з 366 illicit-tx, найдовший — 182 кроки. Поріг надто загальний

Цикли (1,8 с): приклад проходить через хаб-гаманець з 11 213 відправниками (licit) — round-trip через сервіс, а не обов’язково «відмивання»

Шлях: від illicit-гаманця до головного отримувача illicit-коштів — 2 кроки через інший illicit-гаманець

GRAPH DATA SCIENCE: ЦИКЛ РОБОТИ

1. Проєкція → **2. Оцінка пам’яті** → **3. Алгоритм** → **4. Аналіз у Cypher** → **5. Експорт у Gephi**

```
// 1. Граф у пам'яті: гаманці, неорієнтовано, вага n
CALL gds.graph.project('wallets', 'Wallet',
  {SENT_TO: {orientation: 'UNDIRECTED', properties: 'n'}});

// 2. Скільки пам'яті потрібно?
CALL gds.louvain.write.estimate('wallets',
  {writeProperty: 'louvain'}) YIELD requiredMemory;

// 3. Алгоритм -> властивість у базі
CALL gds.louvain.write('wallets', {writeProperty: 'louvain',
  relationshipWeightProperty: 'n'})
YIELD communityCount, modularity;
```

Режими виконання

- `stream` — подивитися результат
- `stats` — лише підсумок
- `mutate` — у проєкцію, для наступного алгоритму
- `write` — записати у вузли бази

Проєкція живе в пам’яті: після роботи — CALL gds.graph.drop('wallets').

АЛГОРИТМИ GDS ДЛЯ РОЗСЛІДУВАНЬ

Алгоритм	Питання розслідувача	Нюанс
WCC	Які «острови» не пов’язані між собою?	Базова кластеризація; гігантський компонент — норма
Louvain	Хто взаємодіє щільніше, ніж випадково?	Модулярність; параметр resolution змінює розмір спільнот
Leiden	Те саме, але стабільніше	Гарантує зв’язні спільноти; запускайте на неорієнтованій проєкції
Label Propagation	Швидкі спільноти на дуже великих графах	Недетермінований: фіксуйте seed, повторюйте
PageRank	Куди «стікаються» кошти?	Високий у сервісів, колекторів, бірж
Betweenness	Хто посередник між групами?	Дорогий: семплювання або підграф
Degree	Fan-in і fan-out	orientation: 'REVERSE' — вхідний ступінь
FastRP + k-means	Хто схожий за формою сусідства?	Знаходить схожих без спільних ребер

ПЕРЕРВА

5 хвилин

Після перерви:

- Кластеризація адрес і спільноти
- Тренди та дрейф у часі
- Gephi: імпорт, ForceAtlas2, фільтри
- ML-бенчмарк і обмеження
- Практика після заняття

ЧАСТИНА 3

Кластеризація



- Три значення слова «кластер»
- Евристики кластеризації адрес
- CoinJoin: чому його виключають
- Louvain і Leiden
- Результати на Elliptic++ і ліди
- ML-бенчмарк статті

ТРИ ЗНАЧЕННЯ «КЛАСТЕРИЗАЦІЇ»

1

Кластери адрес
Entity resolution

Питання: які адреси належать одному власнику?

Метод: евристики спільних входів і решти

Результат: вузол = сутність (як у BitcoinTemporalGraph)

2

Спільноти в графі
Community detection

Питання: хто взаємодіє щільніше, ніж з рештою?

Метод: WCC, Louvain, Leiden

Результат: групи гаманців або сутностей

3

Кластери ознак
Feature clustering

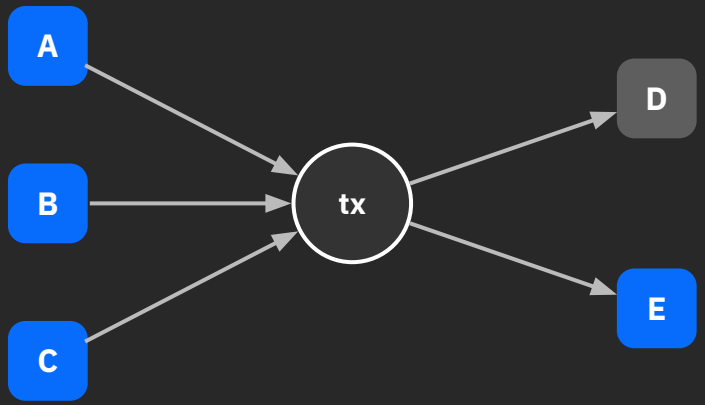
Питання: хто поводиться схоже?

Метод: k-means на ознаках або ембедингах (FastRP)

Результат: поведінкові профілі

Пишіть у звіті, яку саме кластеризацію застосовано: це три різні твердження.

ЕВРИСТИКИ КЛАСТЕРИЗАЦІЇ АДРЕС



Синій = одна оцінена сутність: A, B, C — спільні входи; E — імовірна решта

Спільні входи (common-input ownership)

Усі входи однієї tx підписує один власник → адреси об’єднуються

Адреса решти (change)

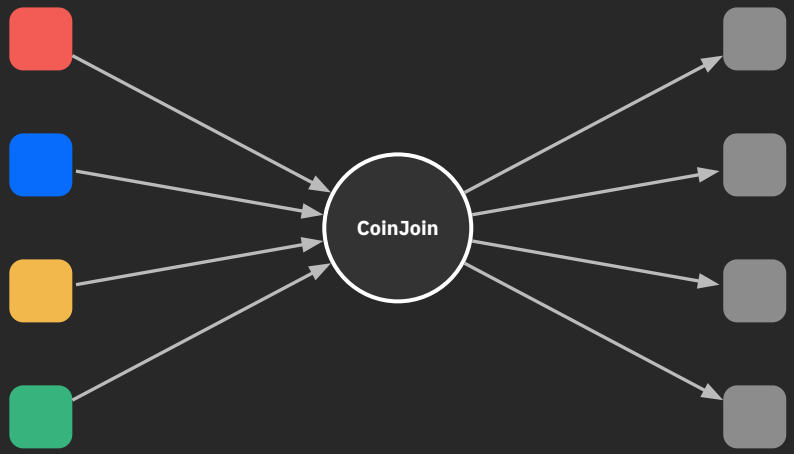
Новий вихід того ж типу, «некругла» сума, адреса з’являється вперше

У статті

Евристики з попередньої роботи авторів (arXiv 2403.00523): 874 млн скриптів → ~252 млн кластерів (у ~3,5 раза менше)

Хибне злиття незворотне: один CoinJoin, пропущений фільтром, «склеює» сторонніх людей в одну сутність.

COINJOIN: ЧОМУ ЙОГО ВИКЛЮЧАЮТЬ



Кольори входів — різні власники; 4 виходи по 0,1 BTC — нерозрізненні

Що відбувається

- Кілька користувачів в одній транзакції
- Рівні виходи ховають, хто кому заплатив
- Евристика спільних входів дає хибне злиття

Що зробили автори

- Детекція евристиками для JoinMarket, Wasabi, Whirlpool (arXiv 2311.12491)
- Виключено і з кластеризації, і з ребер графа

CoinJoin — сигнал бажання приватності, а не доказ злочину.

СПІЛЬНОТИ: LOUVAIN І LEIDEN

Модулярність Q

Частка ребер усередині спільнот мінус очікувана при випадкових зв’язках

Louvain (Blondel et al., 2008)

Жадібно переносить вузли між спільнотами й «згортає» граф рівнями

Leiden (Traag et al., 2019)

Додає крок уточнення: спільноти гарантовано зв’язні

Resolution γ

Більше γ → більше дрібних спільнот

```
CALL gds.louvain.write('wallets', {
  writeProperty: 'louvain',
  relationshipWeightProperty: 'n'})
YIELD communityCount, modularity;

CALL gds.leiden.write('wallets', {
  writeProperty: 'leiden',
  relationshipWeightProperty: 'n',
  gamma: 1.0})
YIELD communityCount, modularity;
```

Біржі й великі сервіси — «супервузли»: вони склеюють спільноти. Позначте або виключіть їх перед запуском.

РЕЗУЛЬТАТ НА ELLIPTIC++

2
компоненти WCC

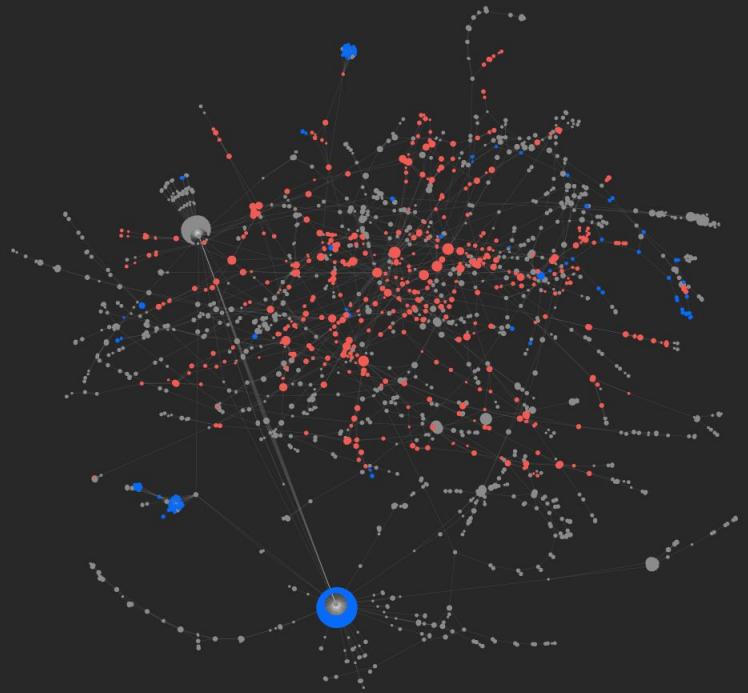
450
спільнот Louvain

0,927
модулярність Louvain

428
спільнот Leiden

Спільнота	Розмір	Illicit	Illicit, %
617545	3 985	1 123	28,2
323119	2 448	501	20,5
512119	4 694	894	19,0
670389	2 616	473	18,1
137179	1 397	239	17,1

Базова частка illicit серед гаманців — 1,7%. Лідер — 28,2%: у 16 разів вище



Спільнота 323119: 2 448 гаманців, 3 399 ребер; illicit 501 — червоні, licit 506 — сині, unknown 1 441 — сірі; розмір — PageRank. ForceAtlas2 (networkx)

ВІД СПІЛЬНОТИ ДО ЛІДА

1. Спільнота	Частка illicit у рази вища за базову (1,7%): поріг ≥ 10%. На Elliptic++ максимум — 28%, а не 50%
2. Кандидати	Невідомі гаманці всередині, ранжовані за PageRank. На Elliptic++: 10 спільнот ≥ 10% → 31 811 кандидатів; топ-1 — 1781T...mqt, він же лідер експозиції
3. Патерни	Fan-in, peel chain, експозиція навколо кандидата — чи є структурне підтвердження?
4. Офчейн	Пошук адреси: форуми (BitcoinTalk), мітки оглядачів, санкційні списки OFAC, звіти
5. Висновок	Рівень впевненості, альтернативні пояснення, обмеження даних

Поріг рахуйте від базової частки, а не «на око». Лід — адреса для перевірки, а не підозрюваний.

ЧИ ПЕРЕДБАЧАЄ ГРАФ ТИП СУТНОСТІ?

Модель	Macro-F1	Exchange	Mining	Gambling	Ponzi	Individual	Ransomware	Bet
GCN	0,56	0,43	0,45	0,56	0,46	0,88	0,19	0,98
GraphSAGE	0,62	0,51	0,57	0,58	0,55	0,93	0,25	0,98
GAT	0,64	0,49	0,57	0,62	0,57	0,93	0,32	0,98
GIN	0,63	0,54	0,54	0,62	0,60	0,94	0,21	0,98
GBC (без графа)	0,57	0,54	0,45	0,57	0,30	0,94	0,50	0,89

Задача: за ознаками вузла та його 2-крокового сусідства передбачити один із 7 типів
Сусідство: 10 сусідів на 1-му кроці, 5 на 2-му; 12 вибірок на вузол

Моделі: GNN з PyTorch Geometric (2 шари, 256 нейронів) і GBC зі sklearn
Розбиття: 40 / 30 / 30, класи вирівняні до 300–1500 зразків

Найкраща модель — GAT (macro-F1 0,64): сусідство допомагає. Але ransomware краще знаходить GBC без графа (0,50).

ДЕ ПОМИЛЯЄТЬСЯ МОДЕЛЬ

Справжній ↓ / прогноз →	Exch.	Mining	Gamb.	Ponzi	Indiv.	Rans.	Bet
Exchange	64%	3%	7%	4%	19%	2%	1%
Mining	5%	70%	3%	3%	17%	1%	0%
Gambling	7%	1%	68%	6%	15%	1%	3%
Ponzi	5%	1%	4%	75%	14%	0%	1%
Individual	3%	2%	3%	2%	88%	2%	0%
Ransomware	14%	0%	3%	0%	23%	60%	0%
Bet	0%	0%	1%	0%	0%	0%	99%

Що видно

- Усе «стікає» в Individual: 14–23% вузлів кожного класу, крім Bet
- Ransomware: recall 60%, але F1 0,32 → точність ≈ 0,22: ~4 з 5 «ransomware»-прогнозів хибні
- Bet — 99%: окрема адреса на кожну ставку

Для OSINT: ML-оцінка — це черга перевірки, а не атрибуція. Рахуйте точність, а не лише «скільки знайшли».

ЧАСТИНА 4

Тренди



- Час у двох датасетах
- Illicit за кроками часу
- Дрейф після закриття ринку (t = 43)
- Тренди в Cypher і Timeline у Gephi

ЧАС У ДАНИХ

Elliptic++: кроки

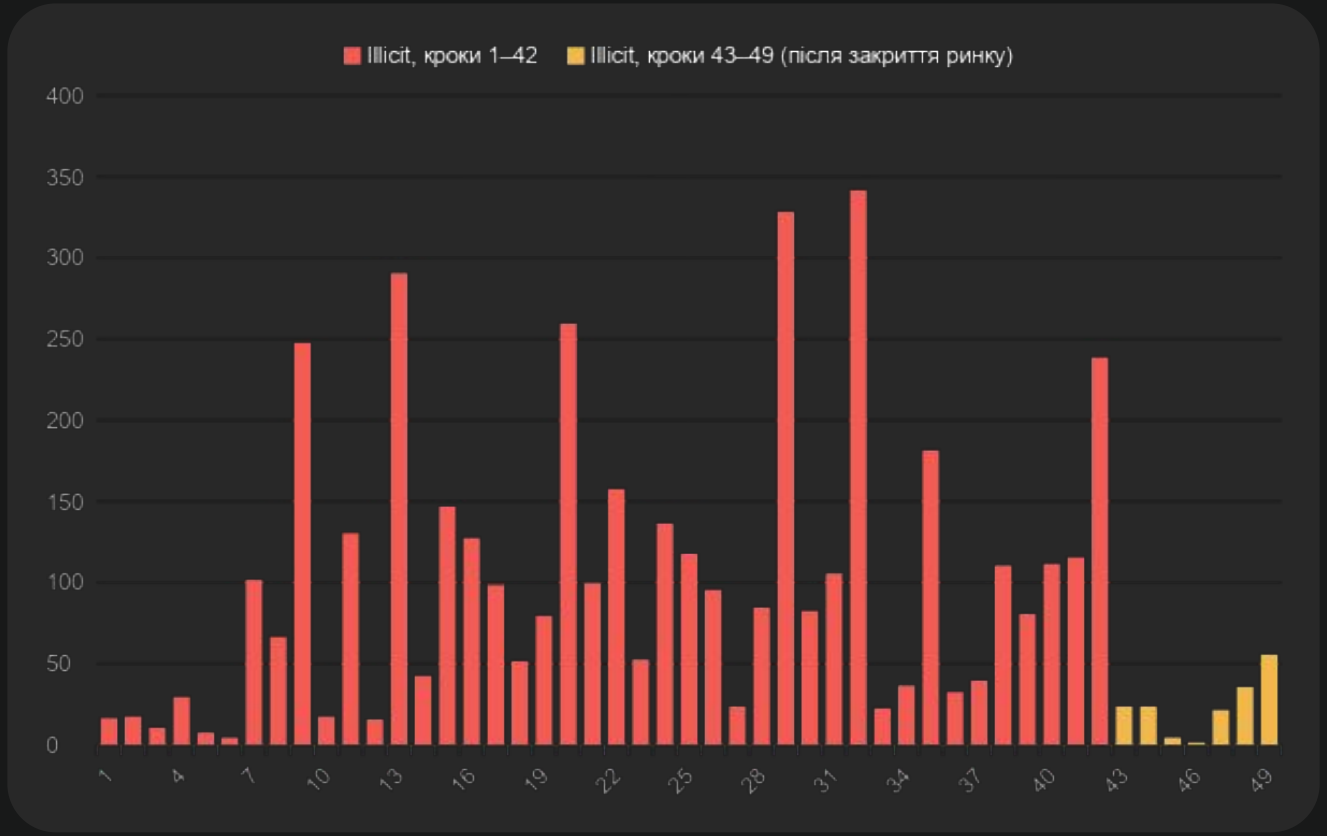
- 49 кроків, ~2 тижні між ними
- Крок — транзакції, підтверджені в межах < 3 годин
- Ребер між кроками немає (перевірено: 49 компонент, 0 ребер між кроками)
- Гаманець: firstStep, lastStep

BitcoinTemporalGraph: блоки

- Мітка часу — індекс блоку (~10 хв)
- Ребро: reveal (перша tx) і last_seen (остання)
- Вузол: перша й остання вхідна та вихідна tx
- Вік = max(last) − min(first)
- Активність ділять на вік; суми → USD за медіанним курсом за період активності

Порівнюйте швидкості, а не абсолютні суми: 1 BTC у 2011 і у 2021 — різні «гроші».

ТРЕНД: ILLICIT ЗА КРОКАМИ ЧАСУ



104

illicit tx на крок, кроки 1–42

24

illicit tx на крок, кроки 43–49

11,0% → 2,5%

частка illicit серед розмічених: кроки 1–42 → 43–49

ДРЕЙФ: ЧОМУ МОДЕЛІ «СЛІПНУТЬ»

Weber et al., 2019 (Elliptic)

- Навчання на кроках 1–34, тест на 35–49 (70 : 30 за часом)
- Крок 43: раптове закриття даркнет-ринку
- Після нього жоден метод не виявляє нові illicit-транзакції
- Навіть Random Forest, перенавчений після кожного кроку

Що це означає для OSINT

- Екосистема змінюється стрибками: закриття ринків, санкції, нові сервіси
- Приклад з воркшопу: Garantex → A7A5 → Grinex
- Старі патерни застарівають — перевіряйте якість по кроках часу, а не в середньому
- Стежте за часткою unknown і оновлюйте мітки

Тренд без контексту подій — лише крива. Накладайте на шкалу реальні події: закриття, арешти, санкції.

ТРЕНДИ: CYPHER І TIMELINE У GEPHI

```
MATCH (t:Transaction)
RETURN t.timeStep AS step, count(*) AS total,
  count(CASE WHEN t.class = 1 THEN 1 END)
    AS illicit
ORDER BY step;
```

```
// Вузли з інтервалом часу для Gephi
CALL apoc.export.csv.query(
 'MATCH (t:Transaction) RETURN t.txId AS Id,
  t.class AS class, t.timeStep AS start,
  t.timeStep + 1 AS end', 'tx_time.csv', {});
```

1. Імпорт вузлів із колонками start, end і ребер
2. Data Laboratory → Merge Columns → Create time interval (Parse numbers)
3. Увімкнути Timeline, задати вікно = 1 крок
4. Play або тягнути вікно: як з’являються й зникають кластери
5. Statistics рахуються для видимого вікна — порівнюйте кроки між собою

ЧАСТИНА 5

Gephi: від підграфа до висновку

- Gephi у 2026: що нового
- Пайплайн Neo4j → Gephi
- Імпорт і статистики
- ForceAtlas2 і Appearance
- Як читати картинку
- Фільтри та візуалізація як доказ



GEPHI У 2026

Gephi Desktop 0.11

- Реліз 0.11 — травень 2026; актуальна версія на gephi.org — 0.11.4
- Новий OpenGL-рушій: до ~10 млн елементів (за заявою розробників)
- Панель Attributes, нові Settings, трансформація Mirror
- Інтерфейс тепер і українською

Gephi Lite 1.0

- Реліз — жовтень 2025, працює в браузері: lite.gephi.org
- ForceAtlas2, Louvain, фільтри, Appearance
- Для невеликих графів: на момент релізу — до ~10 тис. вузлів або 20 тис. ребер
- Зручно, коли немає прав на встановлення

Ролі: Neo4j — зберігання, запити, алгоритми. Gephi — дослідження, layout і візуальна історія для звіту.

NEO4J → GEPHI: ПАЙПЛАЙН

Підграф (Cypher) → **Експорт (APOC)** → **Імпорт у Gephi** → **Statistics** → **Layout + Appearance** → **Preview → SVG / PDF**

```
// A. Его-мережа illicit-гаманця -> GraphML
CALL apoc.export.graphml.query(
  'MATCH p = (:Wallet {address: "1Abc..."})
            -[:SENT_TO*1..2]-(:Wallet)
   RETURN p LIMIT 2000',
  'ego_illicit.graphml', {useTypes: true});

// B. Спільнота -> CSV для Import Spreadsheet
CALL apoc.export.csv.query(
  'MATCH (w:Wallet) WHERE w.louvain = 123
   RETURN w.address AS Id, w.class AS class,
          w.pagerank AS pagerank', 'nodes.csv', {});
CALL apoc.export.csv.query(
  'MATCH (a:Wallet)-[r:SENT_TO]->(b:Wallet)
   WHERE a.louvain = 123 AND b.louvain = 123
   RETURN a.address AS Source, b.address AS Target,
          r.n AS Weight', 'edges.csv', {});
```

Нотатки

- Потрібно apoc.export.file.enabled=true
- Файли з’являються в папці import/
- GraphML зберігає типи властивостей
- CSV: колонки Id; Source, Target, Weight — Gephi розпізнає їх сам
- Его-мережа на 2 кроки навіть з LIMIT 2000 — це 1 319 вузлів (центр має 644 сусіди)

GEPHI: ІМПОРТ І СТАТИСТИКИ

Імпорт	File → Import spreadsheet: спершу nodes.csv (Id, class, pagerank), потім edges.csv → Append to existing workspace; граф Directed
Перевірка	Data Laboratory: типи колонок (class — як текст для розфарбування), кількість вузлів і ребер
Ступені	Statistics → Average Degree: in-degree / out-degree = fan-in / fan-out
Спільноти	Statistics → Modularity: Resolution, Use weights, Randomize → Modularity Class
Ролі	PageRank; Network Diameter → Betweenness і Closeness (повільно на великих графах)
Компоненти	Connected Components → Component ID; фільтр Giant Component

GEPHI: FORCEATLAS 2 І APPEARANCE

ForceAtlas 2 (Jacomy et al., 2014)

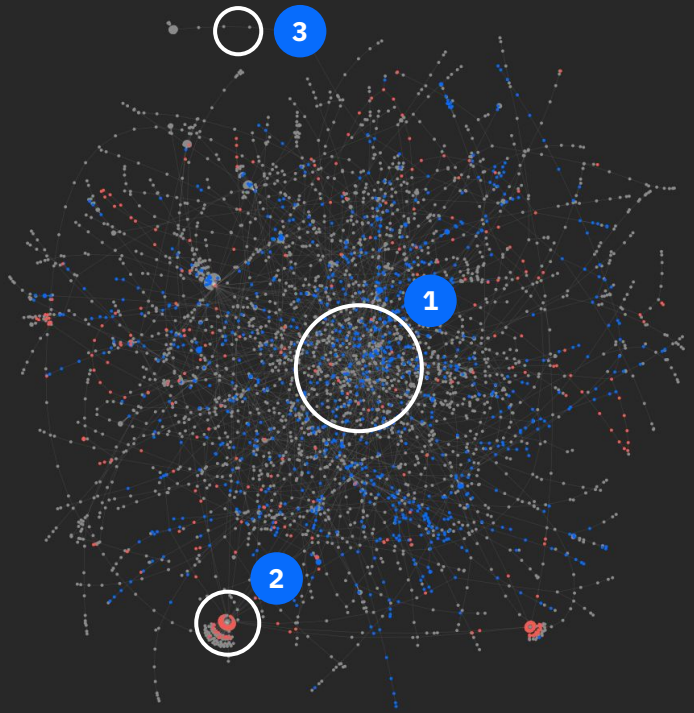
- **Scaling** — розтягує граф
- **Gravity** — тягне до центру, тримає острови поруч
- **LinLog mode** — компактніші кластери
- **Dissuade Hubs** — хаби на периферію
- **Prevent Overlap** — вмикати наприкінці
- **Approximate Repulsion** (Barnes-Hut) — для великих графів

Appearance

- **Колір вузлів:** Partition → class або Modularity Class
- Палітра: illicit — червоний, licit — синій, unknown — сірий
- **Розмір:** Ranking → PageRank або In-Degree
- **Ребра:** товщина за Weight, колір за джерелом
- **Підписи:** лише для топ-N вузлів

Однакові параметри layout і палітра для всіх рисунків звіту — інакше картинки непорівнювані.

ЯК ЧИТАТИ КАРТИНКУ



Elliptic++, крок 32: 4 525 tx, 4 952 ребра; illicit 342 — червоні, licit 981 — сині, unknown 3 202 — cipi. ForceAtlas2 (networkx)

1 **Щільне ядро** → спільнота

2 **Зірка** → хаб: fan-in / fan-out

3 **Довга нитка** → ланцюг, peel chain

4 **Вузол між групами** → міст, посередник

5 **Острів** → окрема компонента

GEPHI: ФІЛЬТРИ ДЛЯ РОЗСЛІДУВАННЯ

Фільтр	Навіщо
Topology → Ego Network	Оточення ліда на глибину 1–2
Topology → Giant Component	Прибрати дрібні острови
Topology → Degree Range	Відсікти листки або супервузли бірж
Topology → K-core	Залишити щільне ядро
Attributes → Partition	Лише class = 1 (illicit) або одна спільнота
Attributes → Range	PageRank або Weight вище порогу
Dynamic → Time Interval	Вікно часу (Timeline): один крок або кілька

Фільтри комбінуються (перетягніть один в інший). Фільтр — частина методології: записуйте, що відкинуто.

ВІЗУАЛІЗАЦІЯ ЯК ДОКАЗ

1. Легенда кольорів і розмірів
2. Джерело даних, дата, версія датасету
3. Що відфільтровано і чому
4. Параметри layout і збережений проєкт .gephi
5. Мітки — це дані датасету, а не встановлені факти
6. Експорт з Preview у SVG або PDF (вектор)

Картинка має бути відтворюваною: інший аналітик з тими самими даними отримує ту саму структуру.

ОБМЕЖЕННЯ ТА ЕТИКА

Дані

- Мітки з форуму можуть бути хибними або дезінформацією
- Лише англомовний розділ BitcoinTalk
- LLM помиляється на короткому контексті
- unknown ≠ licit

Методи

- Евристики дають хибні злиття (CoinJoin, кастодіальні сервіси)
- Супервузли бірж спотворюють спільноти
- Дрейф у часі та дисбаланс класів

Висновки та люди

- Близькість ≠ причетність
- «Узгоджується з…» + рівень впевненості
- Individual-мітки з профілів — персональні дані: мінімізуйте, не публікуйте

Для кожного патерну — альтернативне пояснення. Детальніше про упередження — на окремому занятті курсу.

ПРАКТИКА ПІСЛЯ ЗАНЯТТЯ

1. Розгорнути	Neo4j + GDS + APOC, імпорт Elliptic++ скриптами заняття; знімок підрахунку вузлів і ребер
2. Патерни	По одному прикладу fan-in, peel chain і експозиції: запит, результат, легітимне пояснення
3. Спільноти	Louvain і Leiden, таблиця «чистоти»; обрати спільноту з часткою illicit ≥ 10% і порівняти з базовою
4. Gephi	Візуалізація обраної спільноти: ForceAtlas2, колір = клас, розмір = PageRank, легенда
5. Висновок	1 сторінка: 3 ліди, рівні впевненості, альтернативи, обмеження даних

Здати: файли .cypher, проєкт .gephi, рисунок SVG/PNG і PDF з висновком.

ВИСНОВКИ

Патерни: fan-in, fan-out, peel chain, цикли, CoinJoin, мости — Cypher-шаблони з порогами, які треба обґрунтувати

Кластеризація: розрізняйте кластери адрес, спільноти (Louvain, Leiden) і поведінкові кластери (k-means)

Тренди: час — окремий вимір; нормалізуйте за віком і курсом, перевіряйте дрейф після подій

Інструменти: PostgreSQL — сховище, Neo4j — запити й алгоритми, Gephi — дослідження та візуальний аргумент

OSINT: мітки — це офчейн-розвідка; будь-яке джерело, навіть наукову статтю, перевіряйте

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ